Study Tools: How to Add a New Tab

Developer handoff for scripturelearning.org. Written October 16, 2026.

1. How the page works

- The Study Tools page uses the WordPress page template `studytools.php`, at `/var/www/html/wp-content/themes/astra-child/page-templates/studytools.php`. It draws the purple banner, then prints the page content with `the_content()`.
- Everything below the banner (tab bar, panels, styles and scripts) is **one Custom HTML block** in the page, edited in the WordPress editor under Pages > Study Tools.
- The template gives the block's scripts two globals: `window.SL_READER_URL` (address of the Bible reader page) and `window.SL_NONCE` (a WordPress REST nonce).
- There is no WordPress menu or plugin behind the tabs. They are plain HTML, CSS and JavaScript.

2. Anatomy of the block

```
<div class="sl-explore" id="slStudy">
  <div class="sl-inner">
    <header class="sl-head">...</header>
    <nav class="sl-tabs" role="tablist">
      <button class="sl-tab is-active" role="tab" aria-selected="true" data-
panel="maps">
        <span class="sl-tab-name">Bible Maps</span>
        <span class="sl-tab-sub">Where it happened</span>
      </button>
      ... one button per tab ...
    </nav>
    <section class="sl-panel is-active" id="panel-maps" role="tabpanel">...
</section>
    <section class="sl-panel" id="panel-books" role="tabpanel" hidden>...
</section>
    ...
    <footer class="sl-foot">...</footer>
  </div>
</div>
<style>...</style>
<script>...</script>
```

- A button's `data-panel="x"` matches the section with `id="panel-x"`.
- Only one panel shows at a time. The visible one has the class `is-active`, and every other panel has the `hidden` attribute.
- The original tabs are Bible Maps, The 66 Books, Kings & Prophets, Family Trees, Covenants, Reading Tracker, Weights & Measures and Ten Commandments. Novenas and The Rosary were added later as add-ons. The cross-reference explorer was built the same way and now also runs as its own page.

3. How tab switching works

The main script keeps a `panels` object (a name mapped to a section). When a tab is clicked it removes `is-active` and `aria-selected="true"` from every tab, marks the clicked tab, then shows the matching panel and hides the others.

Important: it only knows about the panels listed in that object. Panels created by add-ons are not in it, so every add-on must hide itself when a different tab is clicked. The skeleton in section 5 does this.

4. Design tokens and reusable classes

Variables defined on `.sl-explore` (use them as `var(--name)`):

- Colors: `--paper` #FBFAF6, `--card` #FFFFFF, `--ink` #22303B, `--soft` #5A6771, `--line` #E8E3D8, `--line2` #EFEBE1, `--blue` #35618C, `--blue-deep` #2A4E73, `--wash` #EEF3F8, `--gold` #B08A3E, `--gold-wash` #F6F0E2.
- Fonts: `--serif` (EB Garamond, for headings) and `--sans` (Inter, for body text). Both load from Google Fonts at the top of the block.

Classes from the main styles that are safe to reuse: `sl-panel-note` (centered intro line), `sl-nowrap`, `sl-h3`, `sl-body`, `sl-fork-note`, and `sl-map-pill` with `is-active` (pill buttons).

Reset warning: `.sl-explore *` sets `margin:0; padding:0; box-sizing:border-box`, so every element starts with no spacing. Set spacing explicitly, and scope all new CSS under `#panel-yourid` so it cannot affect other tabs.

5. Recommended: add the tab as a self-contained add-on

This does not touch any existing code and is easy to remove. Paste it at the very end of the Custom HTML block, after the last `</script>` (and after any add-on it depends on). This skeleton was tested alongside the original tabs and two other add-ons.

```
<style>
#panel-mytool .mt-box{background:var(--card);border:1px solid var(--
line);border-radius:14px;padding:18px 22px;max-width:680px;margin:0 auto}
</style>
<script>
(function(){
  var ID = 'mytool', NAME = 'My Tool', SUB = 'Short subtitle';
  var root = document.querySelector('.sl-explore#slStudy');
  if(!root || root.querySelector('#panel-' + ID)) return;      // guard: run
once
  var nav = root.querySelector('.sl-tabs');
  var footer = root.querySelector('.sl-foot');
  if(!nav || !footer) return;
```

```
  // 1. the tab button
  var tabBtn = document.createElement('button');
  tabBtn.className = 'sl-tab';
  tabBtn.setAttribute('role', 'tab');
  tabBtn.setAttribute('aria-selected', 'false');
  tabBtn.setAttribute('data-panel', ID);
  tabBtn.innerHTML = '<span class="sl-tab-name">' + NAME + '</span><span
class="sl-tab-sub">' + SUB + '</span>';
  nav.appendChild(tabBtn);

  // 2. the panel (inserted before the footer, hidden at first)
  var panel = document.createElement('section');
  panel.className = 'sl-panel';
  panel.id = 'panel-' + ID;
  panel.setAttribute('role', 'tabpanel');
  panel.hidden = true;
  panel.innerHTML = '<p class="sl-panel-note">One-line intro.</p><div
class="mt-box">Content goes here.</div>';
  footer.parentNode.insertBefore(panel, footer);

  // 3. wire it into the existing tab bar
  nav.addEventListener('click', function(e){
    var btn = e.target.closest('.sl-tab');
    if(!btn || !nav.contains(btn)) return;
    if(btn.getAttribute('data-panel') === ID){
      root.querySelectorAll('.sl-tab').forEach(function(x){
x.classList.remove('is-active'); x.setAttribute('aria-selected', 'false');
});
      btn.classList.add('is-active'); btn.setAttribute('aria-selected',
'true');
      root.querySelectorAll('.sl-panel').forEach(function(p){ p.hidden =
true; p.classList.remove('is-active'); });
      panel.hidden = false; panel.classList.add('is-active');
    } else {
      // another tab was clicked: hide this panel and un-highlight this tab
      panel.hidden = true; panel.classList.remove('is-active');
      tabBtn.classList.remove('is-active'); tabBtn.setAttribute('aria-
selected', 'false');
    }
  });
})();
</script>
```

Rules to keep:

- Use a **unique ID** for every tab. The panel id is always `panel-` plus that ID.
- Keep the **guard** line so the code does not run twice.
- Keep the **else branch** in the click handler. Without it the panel stays on screen when another tab is clicked.
- Use one **delegated** click listener on `nav`, as shown. The original tab buttons have their own handlers, and this one works alongside them.
- Place add-ons in order. For example, the Rosary Pray Along add-on looks for the `#panel-rosary` panel, so it must come after the Rosary add-on.

6. Alternative: edit the main block directly

1. Add a `<button class="sl-tab" data-panel="xyz">` to the `<nav class="sl-tabs">` list.
2. Add `<section class="sl-panel" id="panel-xyz" role="tabpanel" hidden>...</section>` before the footer.
3. In the main script, add `xyz: root.querySelector('#panel-xyz')` to the `panels` object.
4. Put the content in the section's HTML, or build it in the script.

This is fine for a simple static tab, but the add-on method in section 5 is safer because it leaves the existing code alone.

7. If the block gets too big: load the code from a file

The Study Tools block is already very large. On this site, saving a large block has sometimes failed to reach the live page, or an old copy was served. If that happens:

1. Put your script in a `.js` file in `/wp-content/uploads/crossrefs/` (upload by SFTP).
2. Add one line at the end of the block: `<script src="/wp-content/uploads/crossrefs/your-file.js?v=1"></script>`
3. Raise the number after `?v=` every time you change the file so browsers fetch the new copy.
4. Inject the CSS from the script by creating a `<style>` element, so the whole tab lives in one file.

`crossref-tab.js` in that folder is a working example. It also supports a standalone page mode when the page contains `<div id="slVerseRefs"></div>`.

8. Data and endpoints available

- **Bible text:** `GET /wp-json/bible-study/v1/chapter?book=<slug>&chapter=<n>&version=KJV` returns `[{"verse":"1","text":"..."}, ...]`. The reader offers KJV, ASV and APO (Apocrypha).
- **Book list:** `GET /wp-json/bible-study/v1/books?version=ALL` returns an array of books with `slug`, `name`, `chapters` and `testament` (`OT`, `NT` or `APO`). Use it to get the exact slug for each book instead of guessing.
- **Reading progress:** `/wp-json/bible-study/v1/progress-all`. Send the header `X-WP-Nonce` with `window.SL_NONCE`.
- **Cross-reference data:** `/wp-content/uploads/crossrefs/<Book>.json`, with keys like `John.3.16`.
- **AI explanations:** `POST /wp-json/scripture/v1/explain` (see the Site Reference doc).
- **Browser storage:** the Novenas tab saves progress in `localStorage` (keys `sl_nov_done_v1` and `sl_nov_intent_v1`), so it is private to each browser.
- **Never put API keys** in the block or in a JavaScript file. Anything needing a key must call a server endpoint.

9. Gotchas

- **Update twice.** In the block popup, click Update, then also click Update on the page itself.
- **Cloudflare caches pages.** After saving, purge the cache and test in a private window. Adding a new `?test=` value to the address helps avoid stale copies.
- **Confirm the save reached the live page.** View the page source and search for a unique string from your code (for example your tab's ID).
- **Check the browser console** (F12 > Console) for red errors. Log a line such as `[mytool] started` in your script so you can tell whether it ran.
- **Mind the page layout.** The tab bar is sticky and wraps onto extra rows, with two tabs per row on narrow phones, so keep tab names and subtitles short.

- **Write the text for the page.** Panels use the Garamond and Inter styling already, so reuse the tokens in section 4 rather than adding new fonts or colors.

10. Test checklist

1. The new tab appears at the end of the tab bar.
2. Click every tab, including the new one: exactly one panel is visible and exactly one tab is highlighted each time.
3. Switch away from the new tab and back, and confirm the content is still correct.
4. Resize to phone width and check the layout.
5. Test in a private window after purging Cloudflare.
6. Open the console and confirm there are no red errors.

11. Existing add-ons to use as examples

- **Novenas:** a tab with a picker, nine day buttons, saved progress and a personal intention field.
- **The Rosary:** a tab with the four sets of mysteries and the full prayers. A separate Pray Along add-on attaches an animated bead diagram and optional audio to this tab, and it must be placed after it.
- **Cross-References:** the verse explorer, kept in `/wp-content/uploads/crossrefs/crossref-tab.js`. It shows how to load external data, call the reader routes and call a server endpoint.